

Proyecto Integrador Evolutivo - PIE

Etapa II — Optimización del sistema mediante Arreglos, Vectores y Matrices

Curso de Algoritmos

Etapa II — Optimización del Sistema

Propósito de la segunda etapa

En la **Etapa I** del Proyecto Integrador Evolutivo se desarrolló la primera versión funcional del sistema utilizando los conocimientos estudiados hasta ese momento:

- variables;
- operadores;
- estructuras de decisión;
- estructuras repetitivas;
- procedimientos;
- funciones;
- parámetros.

En esta **Etapa II**, el mismo proyecto deberá evolucionar mediante la incorporación de estructuras que permitan organizar y procesar una mayor cantidad de información.

El estudiante deberá **modificar y ampliar el sistema desarrollado en la Etapa I**, incorporando de manera justificada uno o más de los siguientes recursos:

- **arreglos;**
- **vectores;**
- **matrices.**

! El proyecto continúa

No debe desarrollarse un proyecto nuevo.

La Etapa II consiste en tomar el sistema construido en la Etapa I y mejorarlo utilizando los nuevos conocimientos adquiridos durante el curso.

Objetivo de la Etapa II

Optimizar y ampliar el sistema desarrollado en la Etapa I mediante el uso adecuado de arreglos, vectores o matrices, integrándolos con variables, decisiones, ciclos, procedimientos, funciones y parámetros.

Al finalizar esta etapa, el estudiante deberá demostrar que su sistema puede **manejar conjuntos de datos**, realizar operaciones sobre ellos y presentar resultados de manera organizada.

¿Qué debe cambiar respecto a la Etapa I?

En la primera versión era posible trabajar con información individual.

Por ejemplo:

```
string nombre;  
float nota;
```

En esta etapa el sistema deberá ser capaz de manejar **varios registros o varios valores relacionados**.

Por ejemplo:

```
string nombres[35];  
float notas[35];
```

o, cuando el problema requiera dos dimensiones:

```
float notas[35][3];
```

La estructura seleccionada dependerá de las necesidades reales del proyecto.



No se trata de agregar un vector solamente para cumplir

El uso de arreglos, vectores o matrices debe resolver una necesidad concreta del sistema.
El estudiante deberá ser capaz de explicar:

¿Qué problema de la Etapa I estoy resolviendo o mejorando mediante esta estructura?

Ejemplos de evolución

Estos ejemplos son únicamente orientativos.

Registro de estudiantes

Etapa I

El sistema podía trabajar con los datos de un estudiante.

Etapa II

Puede almacenar varios estudiantes utilizando arreglos o vectores paralelos:

```
string carnet[35];  
string nombre[35];  
float nota[35];
```

También podría utilizar una matriz cuando necesite almacenar varias notas por estudiante:

```
float notas[35][3];
```

Inventario de equipos

Etapa I

Se ingresaban y procesaban los datos de un equipo.

Etapa II

El sistema puede mantener varios equipos:

```
string codigo[50];  
string descripcion[50];  
int existencia[50];
```

Esto permitiría realizar búsquedas, actualizar existencias o generar listados.

Control de herramientas

El sistema podría utilizar estructuras paralelas para relacionar:

Código
Nombre de herramienta
Cantidad
Estado

y posteriormente recorrer, buscar o procesar los registros almacenados.

Requerimientos de la Etapa II

El sistema deberá conservar los conocimientos utilizados en la Etapa I e incorporar los nuevos contenidos.

Como mínimo deberá evidenciar:

- variables y constantes cuando correspondan;
- operadores;
- estructuras de decisión;
- estructuras repetitivas;
- procedimientos;
- funciones;
- parámetros;
- **arreglos, vectores o matrices;**
- validación de datos;
- organización modular del programa.

No es obligatorio utilizar simultáneamente arreglos, vectores y matrices.

La estructura utilizada deberá ser **adecuada al problema que resuelve el proyecto.**

Fase 1. Actualización del análisis

No es necesario rehacer completamente el análisis de la Etapa I.

El estudiante deberá conservar la descripción original del proyecto y agregar una sección denominada:

Evolución del sistema — Etapa II

Esta sección deberá explicar:

1. ¿Qué podía hacer el sistema en la Etapa I?
2. ¿Qué limitaciones tenía?
3. ¿Qué nueva necesidad se desea resolver?
4. ¿Qué funcionalidad nueva se incorporará?
5. ¿Qué estructura se utilizará: arreglo, vector o matriz?
6. ¿Por qué esa estructura es adecuada para el problema?

Actualización de entradas, procesos y salidas

Deberán identificarse las nuevas entradas, procesos y salidas.

Ejemplo:

Elemento	Etapa I	Etapa II
Entrada	Datos de un estudiante	Datos de varios estudiantes
Proceso	Calcular resultado individual	Registrar, buscar y procesar varios registros
Salida	Resultado individual	Listados, búsquedas y resultados consolidados

Fase 2. Actualización del diseño

Pseudocódigo

El pseudocódigo deberá actualizarse para reflejar las nuevas funcionalidades.

Debe mostrar de manera general:

- declaración de las estructuras;
- ingreso o inicialización de datos;
- recorridos;
- búsquedas o cálculos;
- llamadas a procedimientos y funciones.

No es necesario convertir cada línea de C++ en pseudocódigo.

El objetivo es representar la **lógica de la solución**.

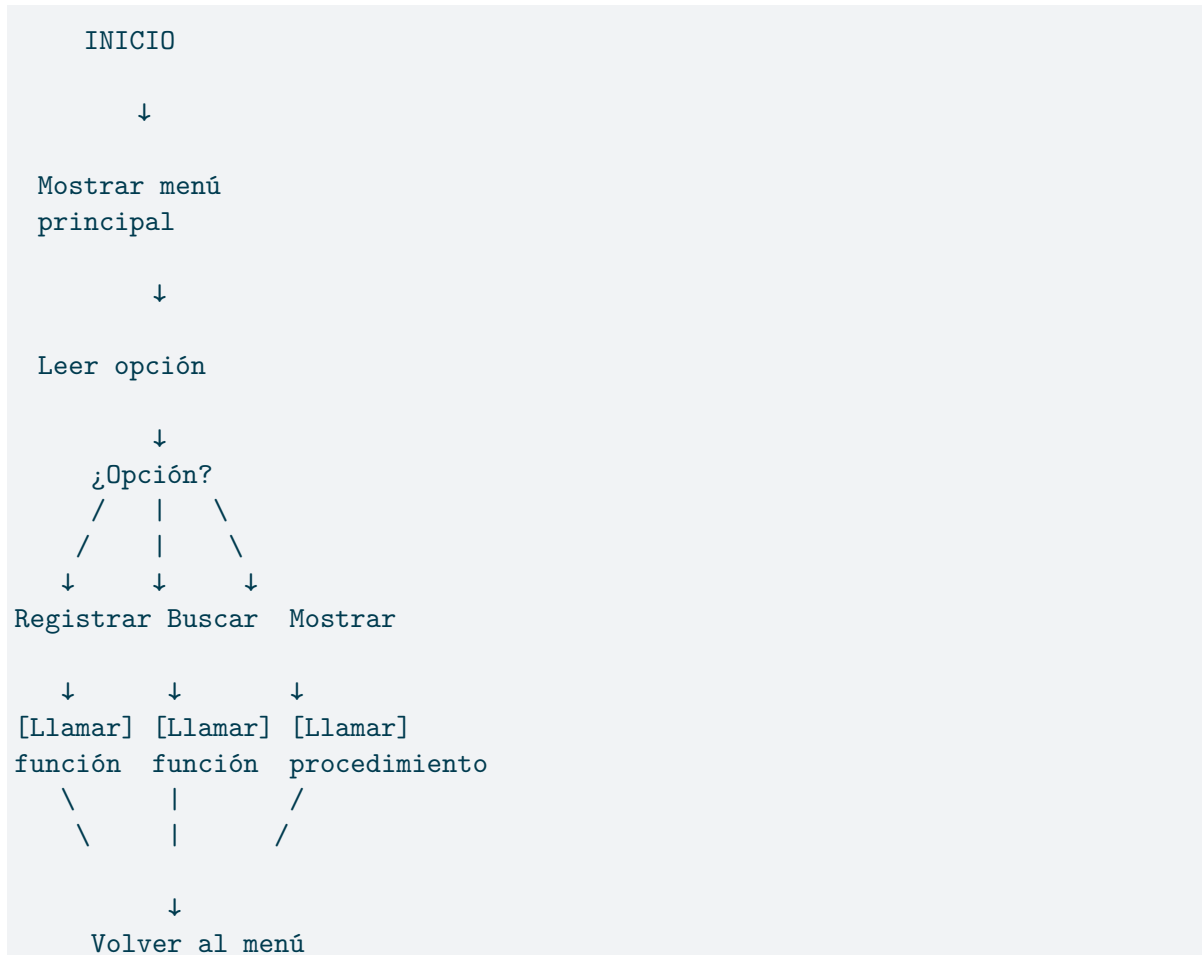
Diagrama de flujo actualizado

El diagrama de flujo de la Etapa I deberá actualizarse.

En esta etapa se busca que el diagrama represente el funcionamiento **general y modular** del sistema.

No es necesario dibujar internamente toda la lógica de cada función.

Por ejemplo:



i Importante sobre el diagrama

Cuando una opción del menú utiliza una función o procedimiento, el diagrama puede representar simplemente la **llamada al módulo**.

No es necesario convertir el diagrama general en un dibujo enorme que contenga inter-

namente todo el código.

Fase 3. Implementación

Esta es la parte principal de la Etapa II.

El estudiante deberá modificar el código fuente de la Etapa I e incorporar las nuevas estructuras de datos.

Uso de arreglos, vectores o matrices

La implementación deberá demostrar que la estructura seleccionada realmente participa en la solución.

No será suficiente declarar:

```
int datos[10];
```

si posteriormente el arreglo no se utiliza.

La estructura deberá permitir realizar operaciones propias del sistema, por ejemplo:

- registrar varios elementos;
- recorrer información;
- buscar un registro;
- contar elementos;
- acumular valores;
- calcular promedios;
- determinar máximos o mínimos;
- mostrar listados;
- relacionar varios vectores mediante la misma posición;
- organizar información mediante filas y columnas.

Integración con funciones y procedimientos

Los arreglos, vectores o matrices deberán integrarse con la modularización aprendida anteriormente.

Por ejemplo:

```
void mostrarNotas(float notas[], int cantidad);
```

o:

```
int buscarEstudiante(string carnet[], int cantidad, string buscado);
```

La idea es evitar concentrar toda la lógica dentro de `main()`.

`main()` deberá funcionar principalmente como coordinador del programa:

```
mostrar menú  
leer opción  
llamar procedimiento o función  
continuar
```

Fase 4. Pruebas de la Etapa II

La segunda etapa deberá incluir **nuevas pruebas**, enfocadas especialmente en las funcionalidades incorporadas.

El estudiante deberá documentar como mínimo **cinco escenarios de prueba**.

Cada prueba deberá incluir:

- objetivo de la prueba;
- datos utilizados;
- resultado esperado;
- resultado obtenido;
- evidencia de ejecución;
- conclusión de la prueba.

Puede utilizarse una tabla como la siguiente:

No.	Prueba	Datos utilizados	Resultado esperado	Resultado obtenido	Estado
1	Registro	Datos válidos	Registro almacenado	Registro almacenado	Correcto
2	Búsqueda existente	Código registrado	Mostrar información	Información encontrada	Correcto
3	Búsqueda inexistente	Código no registrado	Mostrar mensaje	Mensaje mostrado	Correcto

No.	Prueba	Datos utilizados	Resultado esperado	Resultado obtenido	Estado
4	Capacidad	Límite de registros	No exceder capacidad	Sistema controla límite	Correcto
5	Validación	Dato inválido	Rechazar dato	Solicita nuevamente	Correcto

! Las capturas no sustituyen la explicación

No basta con colocar cinco imágenes del programa funcionando.

Cada evidencia deberá indicar **qué se estaba probando y cuál fue el resultado**.

Fase 5. Documentación del código

Se mantienen los requisitos de documentación de la Etapa I.

Cada procedimiento y función deberá incluir comentarios que indiquen:

- propósito;
- parámetros;
- valor de retorno, cuando corresponda.

Ejemplo:

```
// Busca un carnet dentro del arreglo.
// Parámetros:
//   carnet[]: arreglo que contiene los carnets registrados.
//   cantidad: número de registros utilizados.
//   buscado: carnet que se desea localizar.
// Retorna:
//   posición donde fue encontrado o -1 si no existe.
int buscarCarnet(string carnet[], int cantidad, string buscado) {
    // ...
}
```

También deberán comentarse los bloques principales cuando sea necesario para facilitar la comprensión del programa.

Código fuente que debe entregarse

La Etapa II deberá incluir el **programa completo actualizado**, no únicamente las líneas nuevas.

El docente debe poder observar claramente:

```
ETAPA I
Variables + decisiones + ciclos
+ procedimientos + funciones + parámetros
```

↓ EVOLUCIÓN

```
ETAPA II
Todo lo anterior
+ arreglos / vectores / matrices
+ nuevas operaciones
+ pruebas actualizadas
```

💡 Lo más importante de esta etapa

El código debe demostrar que el estudiante comprendió cómo pasar de trabajar con **datos individuales** a trabajar con **conjuntos de datos organizados**.

Restricciones de la Etapa II

Se mantienen las restricciones relacionadas con temas que todavía no han sido estudiados.

No utilizar:

- variables globales;
- librerías no explicadas en clase;
- programación orientada a objetos;
- punteros;
- estructuras (**struct**);
- archivos;
- temas o funciones no estudiadas.

A diferencia de la Etapa I, **sí está permitido y requerido**, según las necesidades del proyecto, utilizar:

- arreglos;

- vectores;
- matrices.

Buenas prácticas de programación

El programa deberá:

- utilizar nombres descriptivos;
- mantener una adecuada indentación;
- evitar código repetido;
- validar datos ingresados;
- utilizar constantes para tamaños cuando corresponda;
- controlar la cantidad de posiciones utilizadas;
- evitar acceder a posiciones fuera de los límites;
- mantener funciones y procedimientos con responsabilidades claras;
- mantener `main()` organizado y legible.

Introducción del informe

La introducción deberá actualizarse.

No debe entregarse exactamente la misma introducción de la Etapa I.

Debe explicar que el proyecto continúa evolucionando y mencionar brevemente las modificaciones realizadas.

Un ejemplo de orientación sería:

En la primera etapa del Proyecto Integrador Evolutivo se desarrolló una versión funcional del sistema utilizando variables, estructuras de decisión, ciclos, procedimientos y funciones. En esta segunda etapa se amplía la solución mediante el uso de arreglos, vectores o matrices, permitiendo administrar múltiples registros y realizar operaciones sobre conjuntos de datos. La modificación conserva las funcionalidades desarrolladas anteriormente e incorpora nuevas capacidades de almacenamiento temporal, búsqueda, procesamiento y presentación de información.

El estudiante deberá adaptar esta idea a **su propio proyecto**.

Conclusiones de la Etapa II

Las conclusiones deberán corresponder específicamente a lo realizado en esta etapa.

No deben limitarse a frases generales como:

“Aprendimos mucho sobre C++.”

Las conclusiones deberán reflexionar sobre aspectos como:

- qué mejoró respecto a la Etapa I;
- qué problema permitió resolver el arreglo, vector o matriz;
- cómo cambió la organización del programa;
- qué dificultades aparecieron;
- qué demostraron las pruebas;
- qué limitaciones mantiene todavía el sistema.

Como mínimo deberán presentarse **tres conclusiones**.

Estructura de entrega

El documento de la Etapa II deberá presentarse en el siguiente orden:

1. Carátula.
2. Introducción actualizada.
3. Objetivos.
4. Descripción del proyecto.
5. **Evolución del sistema — Etapa II.**
6. Entradas, procesos y salidas actualizados.
7. Pseudocódigo actualizado.
8. Diagrama de flujo actualizado.
9. Código fuente completo y comentado.
10. **Pruebas de la Etapa II.**
11. Evidencias de ejecución.
12. **Conclusiones de la Etapa II.**

Criterios de evaluación

Criterio	Puntos
Actualización del análisis y evolución del sistema	10
Pseudocódigo actualizado	10

Criterio	Puntos
Diagrama de flujo modular actualizado	10
Implementación y uso correcto de arreglos, vectores o matrices	25
Integración con procedimientos, funciones y parámetros	15
Pruebas y validaciones	15
Documentación del código	5
Presentación del informe y conclusiones	10
TOTAL	100

i Criterio fundamental de evaluación

Un programa más grande no necesariamente obtendrá una mejor calificación. Se evaluará principalmente que la nueva estructura esté **bien seleccionada, correctamente utilizada e integrada con lo aprendido anteriormente.**

Uso de Inteligencia Artificial

Se mantienen los lineamientos establecidos para la Etapa I.

El objetivo del proyecto continúa siendo desarrollar las habilidades de programación del estudiante.

Se permite utilizar herramientas de Inteligencia Artificial para:

- corregir errores ortográficos;
- mejorar la redacción del informe;
- comprender mensajes de error del compilador;
- consultar o reforzar conceptos ya estudiados;
- solicitar explicaciones sobre el funcionamiento de una instrucción ya vista en clase.

No se permite presentar como propio código fuente generado total o parcialmente por herramientas de Inteligencia Artificial.

Durante la revisión, cualquier integrante deberá poder:

- explicar las estructuras utilizadas;
- justificar los tamaños seleccionados;
- explicar los recorridos;
- explicar los índices;
- justificar las funciones y parámetros;
- modificar una parte del código cuando el docente lo solicite.

 Regla sencilla

Si el estudiante no puede explicar o modificar el código que presenta, no puede demostrar que dicho código representa su aprendizaje.

Evolución del Proyecto Integrador

Etapa	Evolución
Etapa I	Sistema funcional utilizando variables, decisiones, ciclos, procedimientos, funciones y parámetros.
Etapa II	Optimización y ampliación mediante arreglos, vectores o matrices, manteniendo e integrando los conocimientos de la Etapa I.
Etapa III	Incorporación de archivos de texto para almacenar información de forma permanente.

La idea detrás de la Etapa II

```
ETAPA I
El sistema funciona
    ↓
pero trabaja principalmente
con información individual
    ↓
APRENDEMOS
ARREGLOS / VECTORES / MATRICES
    ↓
ETAPA II
El mismo sistema ahora puede
administrar conjuntos de datos
    ↓
registrar
buscar
recorrer
calcular
comparar
```

```
mostrar  
  ↓  
el software EVOLUCIONA
```

No estamos construyendo tres proyectos diferentes.

Estamos observando cómo **un mismo software cambia conforme aumenta nuestro conocimiento como programadores.**